Reverse Mode & Backprop

Every gradient, in one backward pass

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

What `loss.backward()` computes

The most-executed line in modern AI

```
loss = f(parameters, data)      # forward pass: ONE number comes out
loss.backward()                 # ← this line. What does it DO?
```

Every model you have ever heard of — chatbots, image generators, speech systems — was trained by a loop whose heart is `loss.backward()`, executed **millions of times per run**.

About 20 milliseconds after the call, `∂loss/∂θ` has appeared for **every one** of the model's parameters. Billions of them. From one pass.

> We will compute the reverse pass by hand, implement a scalar autodiff engine in about 35 lines of Python, and compare it with PyTorch.

Today's one idea

Reverse mode computes the gradient of one output with respect to a million inputs in one backward pass.

That asymmetry — not faster chips, not bigger data — is why deep learning is possible at all.

Three questions, in order:

1. What structure hides inside an expression? → **computation graphs**
2. What flows backward through it? → **adjoints** (the chain rule, organized)
3. Who does the bookkeeping? → **you** (micrograd), then **PyTorch**

Where L10 left us: a cliffhanger

Method	Assessment from L10	Cost for $\nabla f,\ f : \mathbb{R}^n \to \mathbb{R}$
symbolic	exact, but expressions explode	—
finite differences	truncation vs rounding — both lose	n evals, **approximate**
forward-mode AD	exact! carry $(\text{value}, \text{derivative})$ pairs	n **passes** — one per input

Forward mode ended L10 undefeated on **accuracy** — and disqualified on **cost**: a million inputs means a million passes.

Today's fix: don't push derivatives forward from each input. Pull them backward from the one output.

Learning outcomes

By the end of this lecture you will be able to:

1. Draw the **computation graph** of any small expression — nodes, edges, DAG.
2. Run a **forward pass** and a complete **backward pass** by hand, node by node.
3. Define the **adjoint** $\bar{v} = \partial f / \partial v$ and apply "arriving × local, add at branches".
4. Explain the **cost asymmetry**: one pass per input (forward) vs one pass per output (reverse).
5. Implement a working scalar autograd engine — **micrograd** — and verify it.
6. Reproduce everything in PyTorch: `requires_grad`, `.backward()`, `.grad`, `zero_grad()`.

Computation graphs

You already compute in graphs

Evaluate $f(x, y) = (x + y) \cdot x^2$ at $(3, 1)$. Watch what your own hand does:

$$a = x + y = 4, \qquad b = x^2 = 9, \qquad f = a \cdot b = 36$$

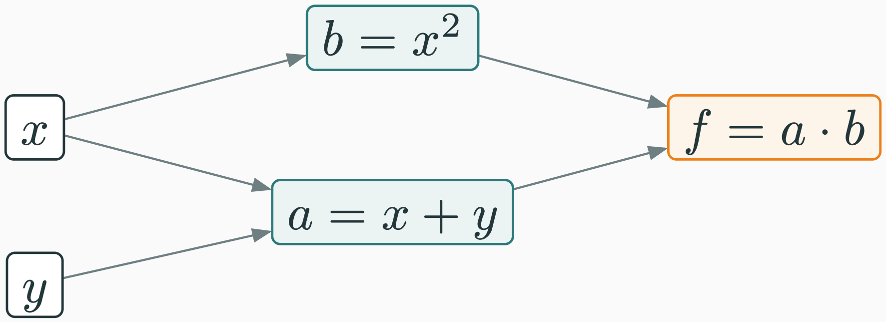
Three small steps, each a **primitive** — a single operation whose derivative we know cold ($+$, $\times$, $(\cdot)^2$, and later $\exp$, $\tanh$, ...).

Each step **names** an intermediate value, and each step **uses** earlier values.

**An expression is a recipe: primitive steps, wired by "who uses whom".
Draw the wiring and you have the computation graph.**

The expression, drawn

V VISUAL



- **Leaves** (left): the inputs x, y. **Root** (right): the output f.
- Every arrow says "this value feeds that operation".

Look at x: it feeds **two** nodes — it is written once but **used twice**. So the picture is a **DAG** (directed acyclic graph), not a tree. Remember this; it is where the one subtle rule of the day will live.

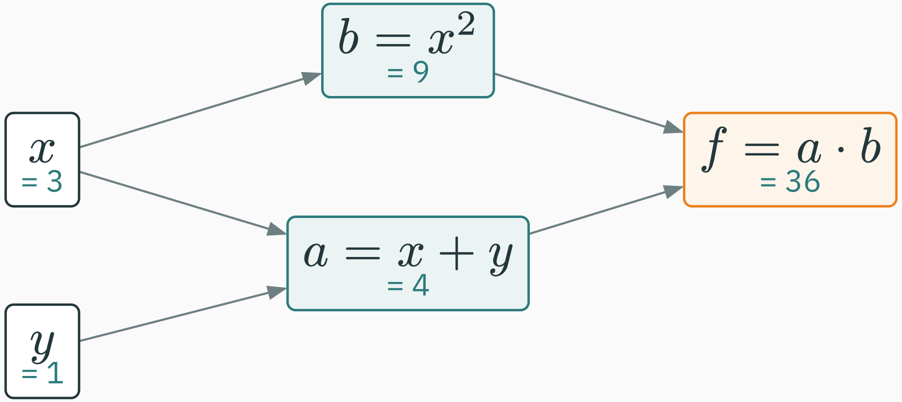
Why graphs? Because computers already build them

- When Python evaluates `(x + y) * x**2`, it **must** execute primitives in some order — the graph is just that execution, remembered.
- PyTorch, JAX, TensorFlow all do exactly this: as your code runs, every operation quietly **records** itself: result, operands, and which primitive fired.
- The recording is called the **tape** (or "autograd graph"). Recording costs almost nothing — the work was being done anyway.

No new math yet: a computation graph is bookkeeping for work the machine already did.

Forward pass: values flow along the arrows

V VISUAL



Node	Rule	Value
$a = x + y$	$3 + 1$	4
$b = x^2$	3^2	9
$f = a \cdot b$	$4 \cdot 9$	36

This is the **forward pass**: leaves are given, every other node fires once its inputs are ready.

Keep every number in sight — the backward pass is about to **reuse all of them**.

Adjoints: the chain rule on a graph

L8's chain rule, re-read as a graph rule

L8 gave the multivariate chain rule. For our $f(a(x,y), b(x))$:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial x}$$

Read it **on the graph**:

> **Multiply local slopes along a path. Add over all paths from the variable to f.**

A deep graph can hold **exponentially many** paths — summing them one by one would be hopeless. Today's algorithm computes each node's "all paths downstream of me" summary **once**, and reuses it. Hold that thought: it is **memoization**.

The adjoint: one number per node

For every node v in the graph, define its **adjoint**

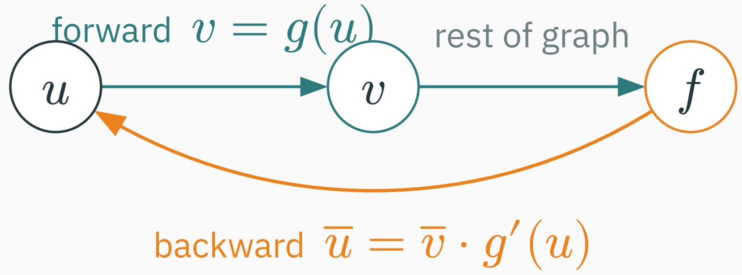
$$\bar{v} := \frac{\partial f}{\partial v} \quad = \quad \text{"if } v \text{ wiggles by } \varepsilon, \;\; f \text{ wiggles by } \bar{v} \cdot \varepsilon\text{"}$$

- The adjoint already accounts for **everything between v and the output** — all paths, summed.
- At the output itself: $\bar{f} = \partial f / \partial f = 1$. Free seed, no work.
- The two adjoints we actually want: $\bar{x}$ and $\bar{y}$ — together they **are** ∇f.

Notation: the bar $\bar{v}$ is standard in the autodiff literature; PyTorch stores exactly this number in `v.grad`.

Each node's job: arriving × local

V VISUAL



The node $v = g(u)$ never sees the whole graph. It receives $\overline{v}$ from above, multiplies by its **own local slope**, and passes the product down:

$$\overline{u} = \overline{v} \cdot \underbrace{g'(u)}_{\text{local, known cold}}$$

$$\boxed{\textbf{derivative sent back} = \textbf{derivative arriving} \times \textbf{local slope}}$$

The three local rules we need today

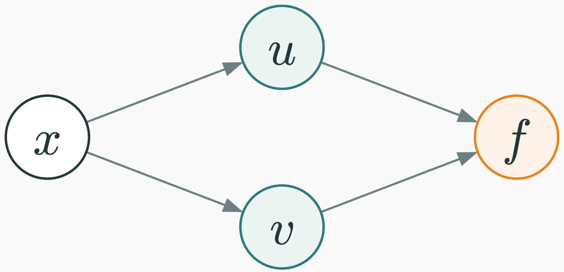
Node	Local slopes	Backward behaviour
$v = u + w$	$\partial v/\partial u = 1, \quad \partial v/\partial w = 1$	$+$ **copies** $\overline{v}$ to both inputs
$v = u \cdot w$	$\partial v/\partial u = w, \quad \partial v/\partial w = u$	$\times$ **sends each input the other input**: $\overline{u} = \overline{v} \cdot w$
$v = u^2$	$\partial v/\partial u = 2u$	square sends $\overline{v} \cdot 2u$

Every primitive an engine supports ships with its one-line rule like these — $\exp$ sends $\overline{v} \cdot e^u$, $\tanh$ sends $\overline{v}(1 - \tanh^2 u)$, ...

An autodiff engine is: this table, plus bookkeeping.

Where paths meet, contributions add

V VISUAL



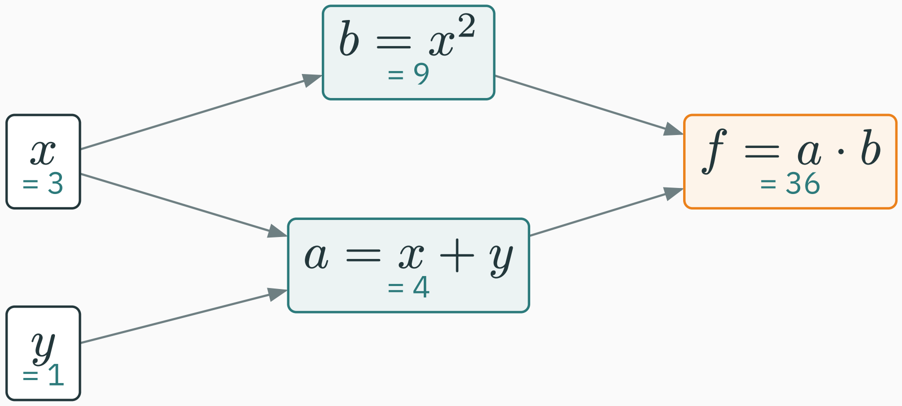
If x feeds two nodes, a nudge Δx travels down **both** routes to f, and the two effects superpose (first-order Taylor — L7):

$$\Delta f \approx \underbrace{\bar{u}\frac{\partial u}{\partial x}}_{\text{via } u} \cdot \Delta x + \underbrace{\bar{v}\frac{\partial v}{\partial x}}_{\text{via } v} \cdot \Delta x \quad \Rightarrow \quad \bar{x} = \text{sum of both contributions}$$

The one subtle rule of the day: at a branch point, backward contributions **accumulate** (+=), never overwrite. Our graph's x is exactly such a point — watch it in the derivation.

⭐ The backward pass, by hand

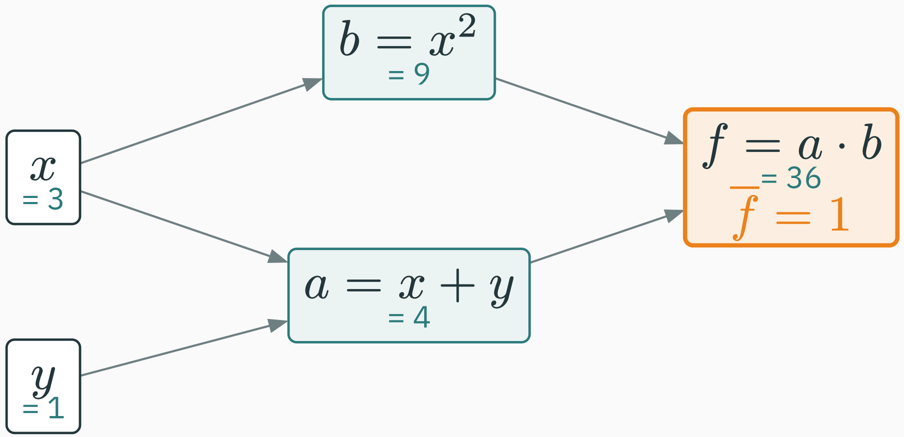
The plan of attack



1. **Seed** the output: $\overline{f} = 1$.
2. Visit nodes from the output back toward the inputs — each node only after **everything it feeds** is done. Here: f, then b, then a.
3. At each node: send **arriving × local** down every incoming edge; where edges meet, **add**.

Step 0 · seed the output

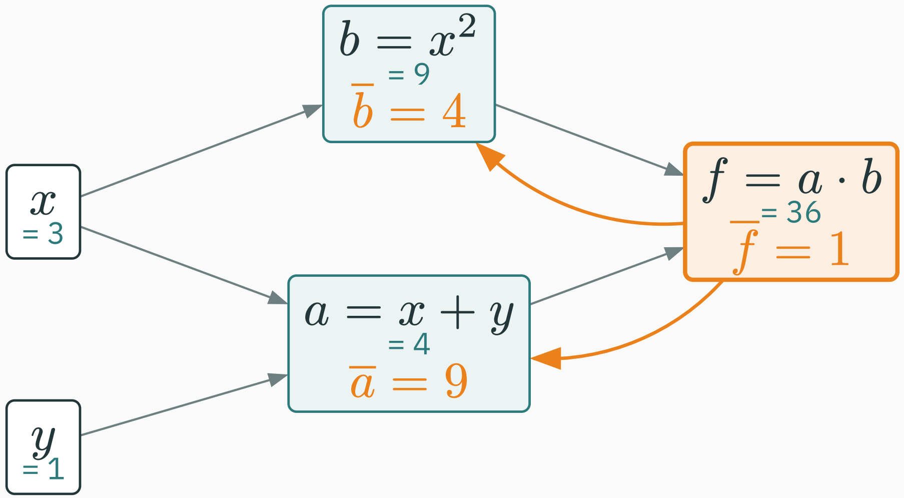
D DERIVATION



$$\bar{f} = \frac{\partial f}{\partial f} = 1 \qquad \text{(a value can't help but move one-for-one with itself)}$$

Every other adjoint will now follow from the local-rules table — no calculus, only arithmetic.

Step 1 · the × node fires



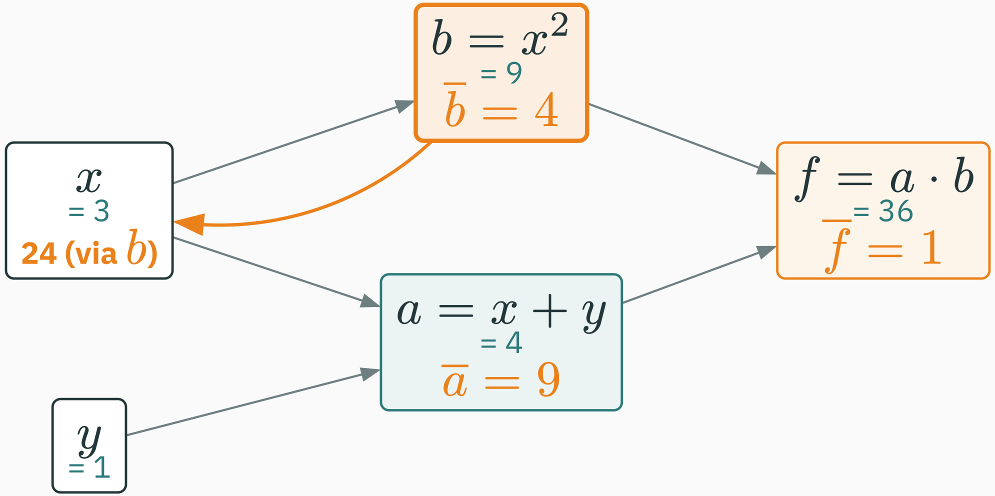
$f = a \cdot b$: **send each input the other input.**

$$\overline{a} = \overline{f} \cdot b = 1 \cdot 9 = 9 \qquad \overline{b} = \overline{f} \cdot a = 1 \cdot 4 = 4$$

With $\overline{f} = 1$, each factor's adjoint **is the other factor's value** — read both off the forward pass, zero new work.

Step 2 · the square node fires

D DERIVATION



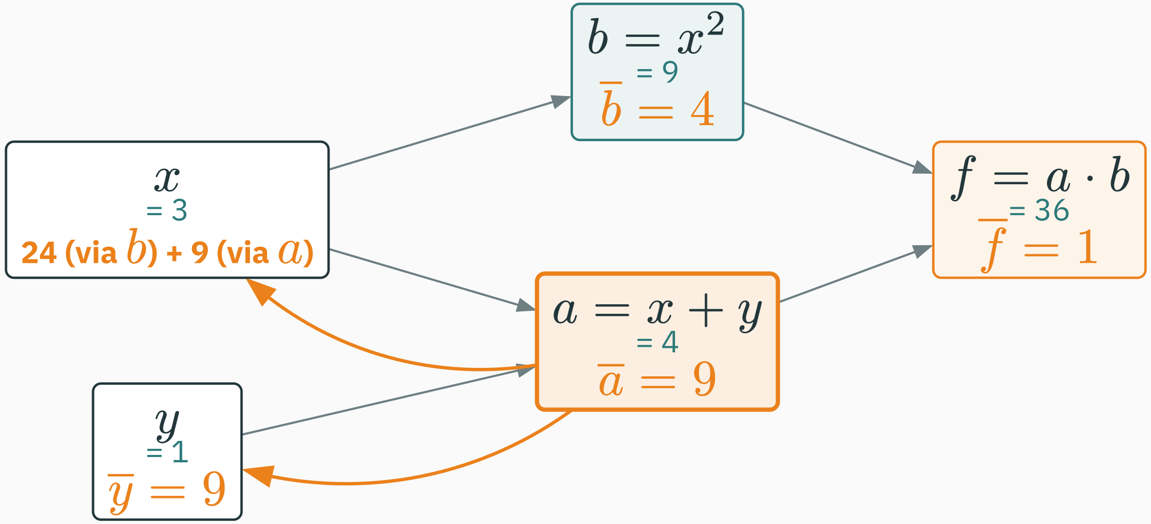
$b = x^2$: local slope $2x = 6$. Arriving × local:

$$\bar{b} \cdot 2x = 4 \cdot 6 = 24 \quad \text{flows down the } b \leftarrow x \text{ edge}$$

Do **not** write $\bar{x} = 24$ yet! A second path into x (via a) has not reported. Park the contribution and wait.

Step 3 · the + node fires

D DERIVATION

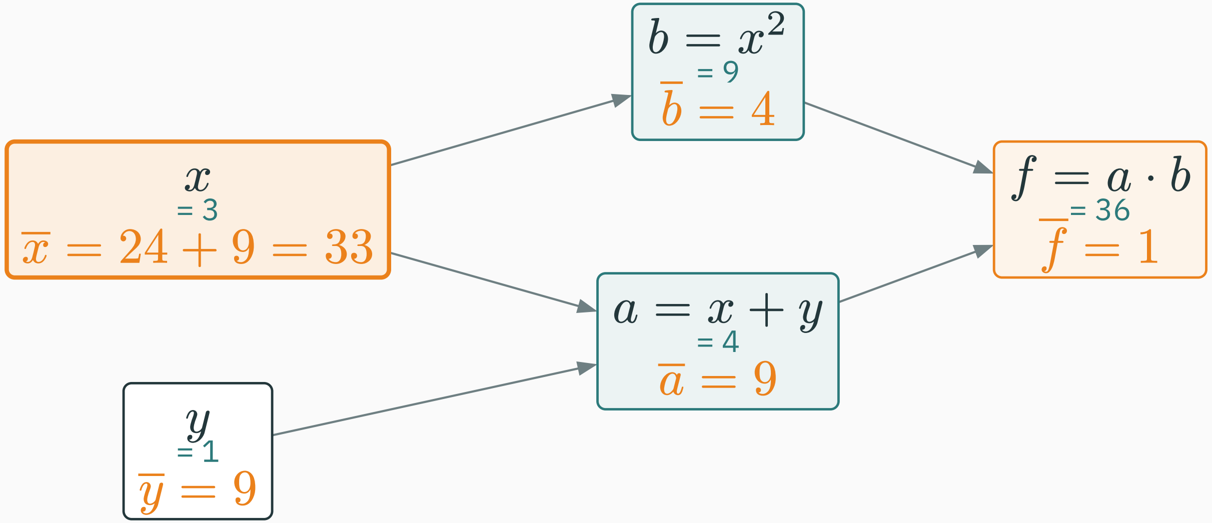


$a = x + y$: **copy the arriving adjoint to both inputs** (local slopes are 1).

$$\text{to} \quad x: \quad \bar{a} \cdot 1 = 9 \qquad\qquad \text{to} \quad y: \quad \bar{a} \cdot 1 = 9$$

y has only this one path, so it is done: $\bar{y} = 9$.

Step 4 · accumulate at the branch



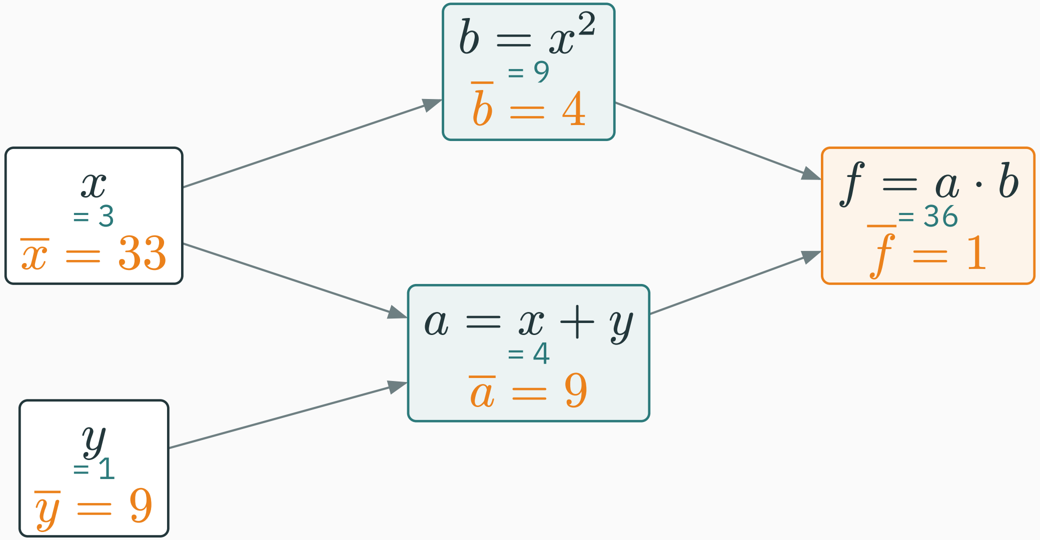
Both of x's paths have reported — the branch rule says **add**:

$$\overline{x} = \underbrace{24}_{\text{via } b} + \underbrace{9}_{\text{via } a} = 33$$

$\nabla f(3,1) = (\overline{x}, \overline{y}) = (33, 9)$ **— every input's derivative, one sweep.**

The finished graph — photograph this

V VISUAL



teal: forward values, flowing right · orange: adjoints, flowing left — same graph, two sweeps

The procedure you just executed has a name: **backpropagation**.

> **Tutorial 5** re-runs this exact graph with fresh numbers until it is reflex; **Quiz 2** assumes it is.

Does calculus agree?

D DERIVATION

Expand and differentiate the old way: $f = (x+y)x^2 = x^3 + x^2 y$.

$$\frac{\partial f}{\partial x} = 3x^2 + 2xy = 27 + 6 = 33 \quad \checkmark \qquad\qquad \frac{\partial f}{\partial y} = x^2 = 9 \quad \checkmark$$

Same numbers. But look **how** each method got there:

- Calculus needed the **global formula** for f, expanded and re-derived per input.
- Backprop only ever used **local** facts — a table of primitive rules and the forward values. Locality is what a computer can automate.

This slide ran a backward pass while compiling

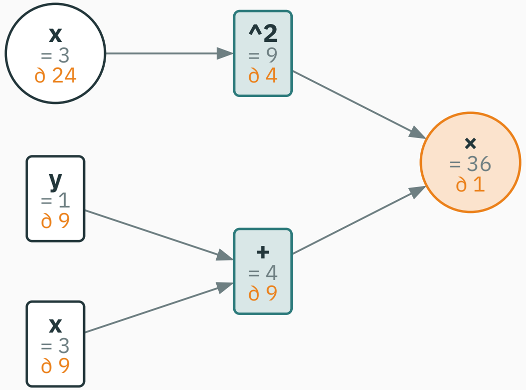
These slides ship their own reverse-mode engine (chalkdust `autodiff`, written in Typst). This slide **runs it** on today's expression — the numbers below are computed live, not typed:

```
#let f5 = ad.expr("(x + y) * x^2", ("x", "y"))
#ad.value(f5, (3, 1))   // forward pass
#ad.grad(f5, (3, 1))    // one backward pass → (x̄, ȳ)
```

engine says: $f = 36, \quad \nabla f = (33, 9)$ **— the slide agrees with your hand.**

Hand, calculus, Typst engine: three routes, one answer. Two more coming (micrograd, PyTorch).

The autodiff tape as a computation graph



auto-drawn from the recorded tape: each node shows op, value (“=”), adjoint (“∂”)

The tape records each **use** of x as its own entry — so the two x leaves carry their per-path contributions 24 and 9 separately. The variable x collects both: $24 + 9 = 33$. The accumulation rule, made visible.

Checkpoint: run the graph at a new point

Q QUESTION

Same graph, new point: $x = 2, y = 5$ — so $a = 7$, $b = 4$, $f = 28$. What is $\overline{y}$?

A. 7

B. 4

C. 28

D. 32

Answer: one path for y

A ANSWER

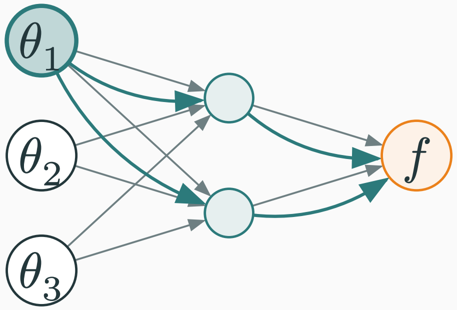
Correct: B — $\overline{y} = 4$

Why: y reaches f through one path: the $+$ node **copies** $\overline{a}$, and the $\times$ node made $\overline{a} = \overline{f} \cdot b = 4$. Sanity check by calculus: $\partial f / \partial y = x^2 = 4$. (Option D is $\overline{x} = 3x^2 + 2xy = 32$ — the **other** input's adjoint.)

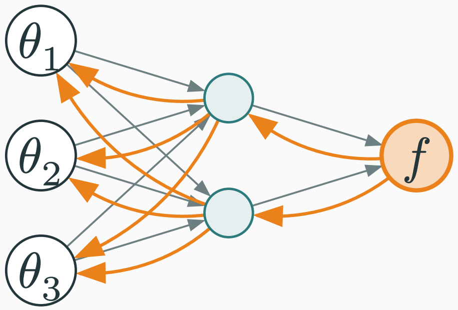
Why reverse mode wins

Two sweep directions, two prices

V VISUAL



forward mode (L10): seed **one input**, push tangents up — yields $\partial f / \partial \theta_1$ only



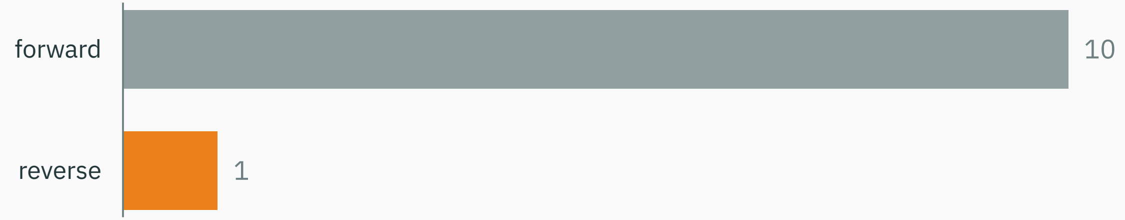
reverse mode (today): seed **the output**, push adjoints down — yields **all** $\partial f / \partial \theta_i$

Forward: one pass per input. **Reverse: one pass per output.**

The scoreboard

V VISUAL

Passes needed to assemble the full gradient of $f : \mathbb{R}^{10} \to \mathbb{R}$:



And the gap **scales**:

Function	**Inputs n**	**Forward-mode passes**	**Reverse passes**
today's $f(x, y)$	2	2	1
small image model's loss	10^7	10^7	1
GPT-3's loss	1.75×10^{11}	1.75×10^{11}	**1**

At 60 ms per pass, forward mode on GPT-3 needs $\approx$ **330 years** per training step.
Reverse: **60 ms**.

ML's shape is exactly reverse mode's shape

Why does reverse mode fit machine learning so perfectly?

- Training tunes n = **millions to billions** of inputs — the parameters θ.
- But it scores them with m = **one** output — the loss $\mathcal{L}(\theta)$ (L14 builds it: a log-likelihood).
- $\mathbb{R}^n \to \mathbb{R}$, gigantic n, $m = 1$: the most extreme "reverse wins" shape that exists.

Millions of knobs, one score. Reverse mode is not an option for ML — it is the option.

Flip the shape and forward mode wins: for $g : \mathbb{R} \to \mathbb{R}^{10^6}$ (one input, many outputs), forward needs 1 pass, reverse 10^6. Rare in ML — but keep it for the checkpoint.

The price: memory (here is the memoization)

Reverse mode is not free. Look what the backward steps **consumed**:

- the $\times$ rule needed the stored values $a = 4$, $b = 9$
- the square rule needed the stored $x = 3$ (for $2x$)

So the forward pass must **keep every intermediate value alive** until backward is done — that stored trail is the tape.

Store each node's value once, reuse it for every path through that node: backprop is the chain rule with memoization.

The trade: forward mode streams in $O(1)$ extra memory; reverse pays $O(\text{graph size})$ memory to win time. This is literally why training a network needs far more GPU memory than running one.

The cheap-gradient principle

How much **time** does the backward sweep add? Each node fires once, doing constant work per edge — the same order of work as the forward pass itself.

Computing ∇f costs a small constant multiple (≈ 2–$3\times$) of computing f — independent of n. True for $n = 2$ and for $n = 1.75 \times 10^{11}$.

This is the **cheap gradient principle** (Baur–Strassen; Griewank). It is why “just take the gradient” is a reasonable instruction at any scale — and why all of modern ML is gradient-based. The catch stays memory, not time.

Checkpoint: pick the mode

Q QUESTION

$f : \mathbb{R}^{10^6} \to \mathbb{R}$ (a loss). $g : \mathbb{R} \to \mathbb{R}^{10^6}$ (one input, a million outputs). Which mode computes all required derivatives cheapest?

A. forward for both

B. reverse for f, forward for g

C. forward for f, reverse for g

D. reverse for both

Answer: match the pass to the thin side

A ANSWER

Correct: B — reverse for f, forward for g

Why: Passes = one per input (forward) or one per output (reverse). f has one output → 1 reverse pass beats 10^6 forward passes. g has one input → 1 forward pass beats 10^6 reverse passes. Always seed the **thin** side.

Micrograd: build the engine

What a number must remember

To automate the hand-run, each value needs to carry exactly what **you** used at the desk:

Field	**What it held in the hand-run**
`data`	the forward value (teal numbers: $3, 1, 4, 9, 36$)
`grad`	the adjoint slot, starting at 0 (orange numbers, `+=` into it)
`_prev`	which nodes fed me (the arrows)
`_backward`	my one line from the local-rules table, frozen as a function

We now write this class in three stages — it is Karpathy's **micrograd**, the engine his assigned video builds. Same fields, same names.

Stage 1 · a `Value` that remembers its history

```
class Value:
    """A scalar that knows how it was computed."""
    def __init__(self, data, _children=()):
        self.data = data                    # forward value
        self.grad = 0.0                     # adjoint: d(output)/d(self)
        self._backward = lambda: None       # local rule (leaves: nothing)
        self._prev = set(_children)         # the nodes that made me
```

- A plain number, plus the graph bookkeeping. `grad = 0.0` so contributions can `+=` in.
- Leaves (x, y) are just `Value(3.0)`, `Value(1.0)` — no children, nothing to send back.

Stage 2a · addition, carrying its backward rule

```
    def __add__(self, other):                    # self + other
        out = Value(self.data + other.data, (self, other))
        def _backward():
            self.grad  += out.grad               # + copies the
            other.grad += out.grad               #   arriving adjoint
        out._backward = _backward
        return out
```

- The forward line computes the value **and wires the graph** (`(self, other)` become `out._prev`).
- The closure is the rules-table row for $+$, waiting: “when my turn comes, copy `out.grad` down”. Note the `+=` — branch accumulation, built in.

Stage 2b · multiplication: send the other input

```
def __mul__(self, other):                    # self * other
    out = Value(self.data * other.data, (self, other))
    def _backward():
        self.grad  += other.data * out.grad   # × sends each
        other.grad += self.data  * out.grad   #   the OTHER input
    out._backward = _backward
    return out
```

- `other.data * out.grad` is precisely **arriving × local** — the local slope of $u \cdot w$ w.r.t. u **is** w.

Each operator owns one closure = one row of the rules table. The engine never needs a global formula.

In what order may nodes fire?

A node may push its `grad` down **only when that `grad` is complete** — after every node it feeds has already fired.

- Fire a before f? Then a pushes `a.grad = 0` — silence — and x, y end up wrong forever.
- Safe order for our graph: f, then b, then a (leaves receive; they never push).
- The general recipe: list nodes so every node comes **after** all nodes it feeds — a **topological order** of the DAG, walked in reverse.

> Getting this order is a classic graph traversal: depth-first, children before self ("post-order"). Ten lines, next slide.

Stage 3 · topological sort, then one sweep

```
def backward(self):
    topo, seen = [], set()
    def build(v):                           # depth-first:
        if v not in seen:
            seen.add(v)
            for c in v._prev:               #   children first,
                build(c)
            topo.append(v)                  #   then me
    build(self)
    self.grad = 1.0                         # seed: df/df = 1
    for v in reversed(topo):                # output → inputs
        v._backward()
```

Seed, sweep, done — steps 0–4 of the hand derivation, mechanized in twelve lines.

Run it on the lecture's graph

```
x, y = Value(3.0), Value(1.0)
f = (x + y) * x * x          # the same 5-node graph
f.backward()

print(f.data)                # 36.0
print(x.grad, y.grad)        # 33.0  9.0
```

$36, 33, 9$ — the hand-run, reproduced by 35 lines of Python you can hold in your head.

We wrote `x * x` because our `Value` has no power op yet. Watch the mul closure: **both** branches are `x`, so it fires `x.grad += x.data * out.grad` twice — the $2x$ rule emerges from accumulation, unasked.

35 lines, and it is real

INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Walk a computation graph in the browser: symbolic vs finite-difference vs autodiff on one screen, forward values then a step-by-step backward sweep. Rebuild today's $f = (x + y) \cdot x^2$ in it and watch 33 and 9 appear.

Assignment 1 (micrograd) goes out this week. You take today's 35 lines and grow them Karpathy's way: `tanh`, `exp`, `__pow__`, `__neg__`, broadcasting scalars — then use **your own engine** to fit a small model. The assigned video walks the full 94-line original; watch it before you start.

PyTorch: the industrial engine

The same numbers, a fifth and final time

```python
import torch

x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(1.0, requires_grad=True)

f = (x + y) * x**2        # forward — PyTorch records the tape
f.backward()              # reverse sweep

print(f.item())           # 36.0
print(x.grad, y.grad)     # tensor(33.)  tensor(9.)
```

Hand · calculus · Typst engine · micrograd · PyTorch: five routes, one gradient $(33, 9)$.

What PyTorch adds to the scalar implementation

Same algorithm as your 35 lines. Different engineering:

- Nodes hold **tensors**, not scalars — one graph node per matrix op, not per number.
- Each primitive (`matmul`, `exp`, `softmax`, ...) ships a hand-tuned backward rule, running on GPU.
- The tape is rebuilt **on every forward run** ("define-by-run") — Python `if`s and loops just work.
- `requires_grad=True` marks which leaves deserve adjoints; everything else is constants.

> That is genuinely all. If you can read your micrograd, you can read `torch.autograd`'s documentation without fear.

Careful: gradients accumulate

`.grad` is an accumulator — `backward()` always `+=`s into it:

```
f = (x + y) * x**2
f.backward()
print(x.grad)           # tensor(33.)

f = (x + y) * x**2      # build the graph again…
f.backward()            # …sweep again
print(x.grad)           # tensor(66.)  ← 33 + 33. Not 33!
```

Why design it this way? The engine cannot know your intent — the same `+=` that merged x's two paths **inside** one graph also merges gradients **across** calls (useful for micro-batching). Clearing is your job.

What `zero_grad()` is for

```
for step in range(1000):
    optimizer.zero_grad()    # 1 · flush stale adjoints
    loss = compute_loss()    # 2 · forward (tape records)
    loss.backward()          # 3 · reverse sweep → .grad
    optimizer.step()         # 4 · nudge parameters (L16!)
```

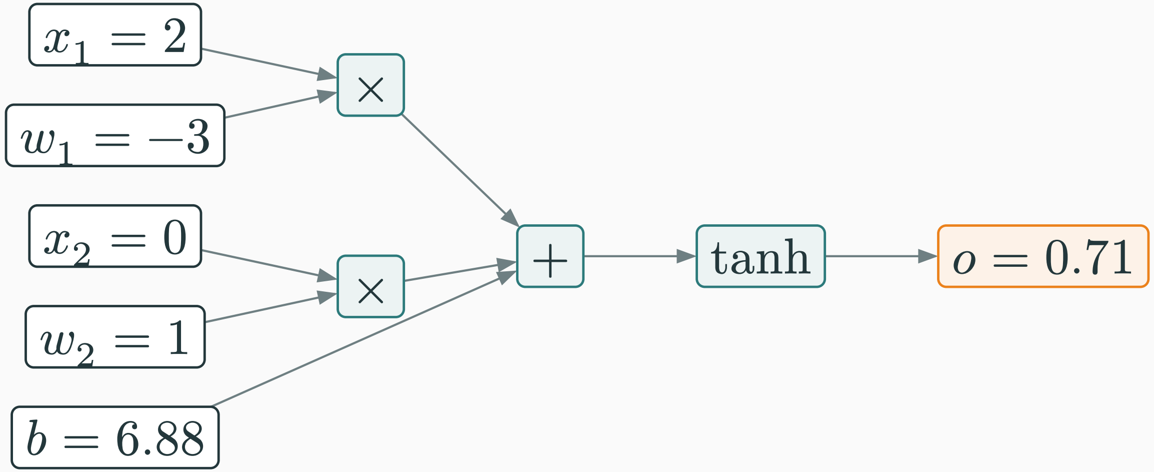
Forget line 1 and nothing crashes — gradients from every past step silently pile up, and training goes sideways. The single most common autograd bug; now you know exactly which `+=` causes it.

Lines 3–4 are the bridge to Module 4: L16 takes the gradients you can now compute and asks **where to step**.

Finale: the graph AI actually trains

One last graph — small algebra, but shaped like the real thing: inputs weighted, summed, squashed.

$$o = \tanh(w_1 x_1 + w_2 x_2 + b)$$



This unit is an (artificial) **neuron** — the atom of every network. To the engine it is nothing special: five leaves, two $\times$, one $+$, one $\tanh$. Backprop needs no new ideas.

Differentiate one neuron with the same engine

```
x1, x2 = torch.tensor(2.0), torch.tensor(0.0)
w1 = torch.tensor(-3.0, requires_grad=True)
w2 = torch.tensor( 1.0, requires_grad=True)
b  = torch.tensor( 6.88137, requires_grad=True)

o = torch.tanh(w1*x1 + w2*x2 + b)    # o = 0.7071
o.backward()
print(w1.grad, w2.grad, b.grad)      # 1.0   0.0   0.5
```

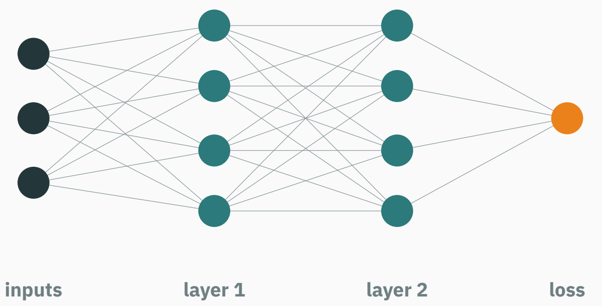
Our Typst engine, live, agrees: $\overline{w}_1 = 1$, $\overline{w}_2 = 0$, $\overline{b} = 0.5$.

Read $\overline{w}_2 = 0$ off the mul rule: $\times$ **sends the other input**, and w_2's partner is $x_2 = 0$. A weight attached to a silent input gets no gradient — no data, no learning signal.

The 20 milliseconds, itemized

Every step of `loss.backward()` is now one you have done yourself:

1. The forward pass already **recorded the tape** (your `_prev` wiring).
2. Topological sort of the tape (your twelve-line `build`).
3. Seed $\overline{\text{loss}} = 1$; sweep once: **arriving × local**, `+=` at branches.
4. Adjoints land in `.grad` — millions at a time.



Bigger graph, same single pass — why deep learning is possible.

Summary & what's next

Lecture 11 — summary

- **Graphs**: an expression is a DAG of primitives; the forward pass fills every value.
- **Adjoints**: $\bar{v} = \partial f / \partial v$; seed $\bar{f} = 1$; each node sends **arriving × local**; branches `+=`.
- **The graph**: $f = (x + y)x^2$ at $(3, 1)$: $f = 36$, $\nabla f = (33, 9)$ — five methods, one answer.
- **Asymmetry**: one pass per input (forward) vs one pass per **output** (reverse); a loss has one output.
- **The price**: the tape stores forward values — memoization; costs memory, not time.
- **Engines**: `data / grad / _prev / _backward` + topo sort + one sweep; in PyTorch `requires_grad` → `.backward()` → `.grad`, with `zero_grad()` every step.

Read before L12 — MML §5.6. **Assigned:** Karpathy, "spelled-out intro to backpropagation" (builds micrograd; watch before starting A1). ★★★ JAX autodiff cookbook. **This week:** T5 drills today's graph by hand · **Quiz 2** (L7–L11) · **A1 (micrograd)** goes out.

Practice problems

Try on paper; verify with the T5 notebook (or your own micrograd).

1. Draw the graph of $g(x, y) = (x - y) \cdot (x + y)$; run forward and backward at $(3, 1)$. Check both adjoints against $g = x^2 - y^2$.
2. Re-run today's graph at $x = 1, y = 2$. Before computing: which of $\overline{x}, \overline{y}$ changes, and why?
3. Give `Value` a `__sub__`. What are its two local slopes, and its two closure lines?
4. In `backward()`, delete `reversed(...)` and trace our graph. What does `x.grad` end as, and which silent push caused it?
5. Predict PyTorch: build and `backward()` the same loss twice with no `zero_grad()` between. What is `w.grad`? Which single line fixes it?
6. For $h : \mathbb{R}^3 \to \mathbb{R}^2$, count passes to get the full Jacobian by each mode. Name a function shape where **forward** mode is the right tool.

⭐⭐⭐ JVPs, VJPs, and Hessians without Hessians

★ OPTIONAL

Both modes are matrix-free products with the Jacobian J of $f : \mathbb{R}^n \to \mathbb{R}^m$ (L9):

Sweep	**Computes**	**Reading**
forward, seeded with $\boldsymbol{v} \in \mathbb{R}^n$	$J\boldsymbol{v}$ — a **JVP**	one **column** mix: sensitivity along direction $\boldsymbol{v}$
reverse, seeded with $\boldsymbol{u} \in \mathbb{R}^m$	$\boldsymbol{u}^\top J$ — a **VJP**	one **row** mix: ∇f is the VJP with $u = 1$

Compose them and curvature comes free: for scalar f, the **Hessian-vector product**

$$H\boldsymbol{v} = \nabla(\nabla f(\theta)^\top \boldsymbol{v}) \quad = \quad \text{forward mode pushed through the reverse-mode program}$$

costs a few function evaluations and **never materializes** the $n \times n$ Hessian — the trick behind Newton-flavoured optimizers (L18) at scales where n^2 storage is fiction. JAX exposes all three as `jvp`, `vjp`, `grad`; see the autodiff cookbook.

Backprop is just the chain rule with memoization.

Next: **Module 3 — Continuous Distributions** — we can now differentiate anything; next we need something worth differentiating: likelihoods.